Directed Acyclic Graph

D. VIJAYAKUMAR
Asst. Prof (Sr. Grade) / CSE
National Engineering College

DAG

- A DAG for a basic block is a directed acyclic graph with the following labels on nodes:

1. Leaves are labeled by unique identifiers, either variable names or constants.
2. Interior nodes are labeled by an operator symbol.
3. Nodes are also optionally given a sequence of identifiers for labels to store the computed values.

- DAGs are useful data structures for implementing transformations on basic blocks.
- It gives a picture of how the value computed by a statement is used in subsequent statements.
- It provides a good way of determining common sub - expressions.

Algorithm for construction of DAG

- **Input**: A basic block
- **Output**: A DAG for the basic block containing the following information:
- 1. A label for each node. For leaves, the label is an identifier. For interior nodes, an operator symbol.
- 2. For each node a list of attached identifiers to hold the computed values. Case (i) x : = y OP z Case (ii) x : = OP y
- Case (iii) x : = y
- **Method:**
- **Step 1:**
- If y is undefined then create node(y).
- If z is undefined, create node(z) for case(i).
- **Step 2:**
- For the case(i), create a node(OP) whose left child is node(y) and right child is node(z). (Checking for common sub expression). Let n be this node.
- For case(ii), determine whether there is node(OP) with one child node(y). If not create such a node.
- For case(iii), node n will be node(y).

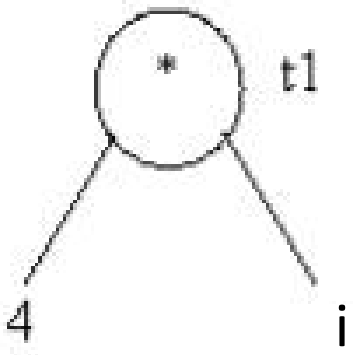
Step 3:

Delete x from the list of identifiers for node(x). Append x to the list of attached identifiers for the node n found in step 2 and set node(x) to n.

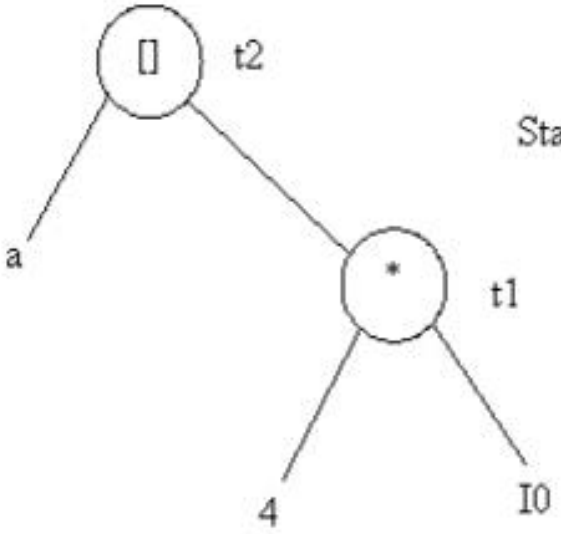
DAG Construction

```
1.  t1 := 4* i
2.  t2 := a[t1]
3.  t3 := 4* i
4.  t4 :=  b[t3]
5.  t5 := t2*t4
6.  t6 := prod+t5
7.  prod := t6
8.  t7 := i+1
9.  i := t7
10. if i<=20 goto (1)
```

1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)

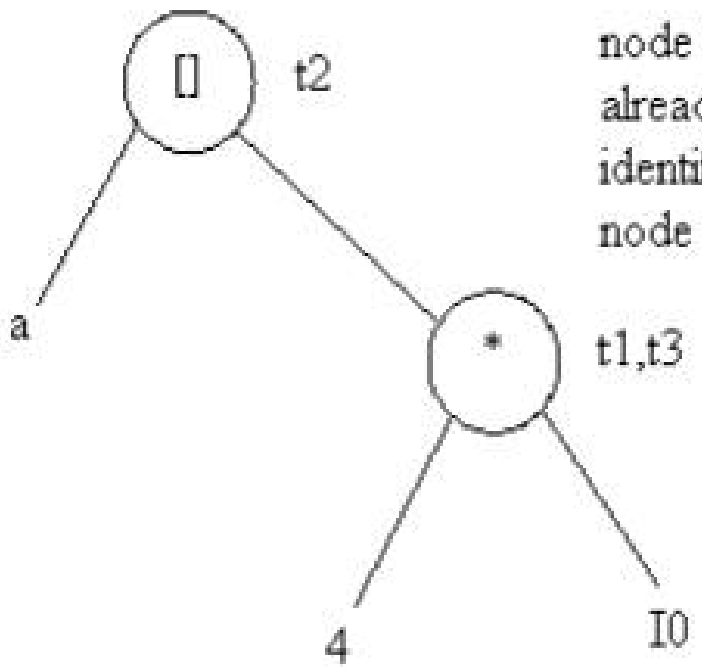


Statement (1)



Statement (2)

1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)



node for 4*I0 exist already, hence attach identifier t3 to the existing node for Statement (3)

1. t1 := 4* i

2. t2 := a[t1]

3. t3 := 4* i

4. t4 := b[t3]

5. t5 := t2*t4

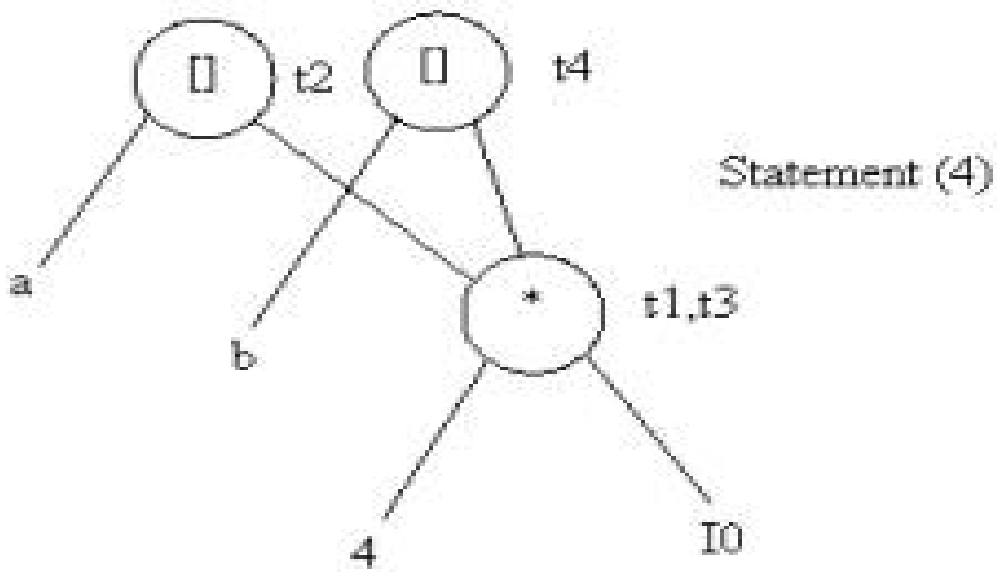
6. t6 := prod+t5

7. prod := t6

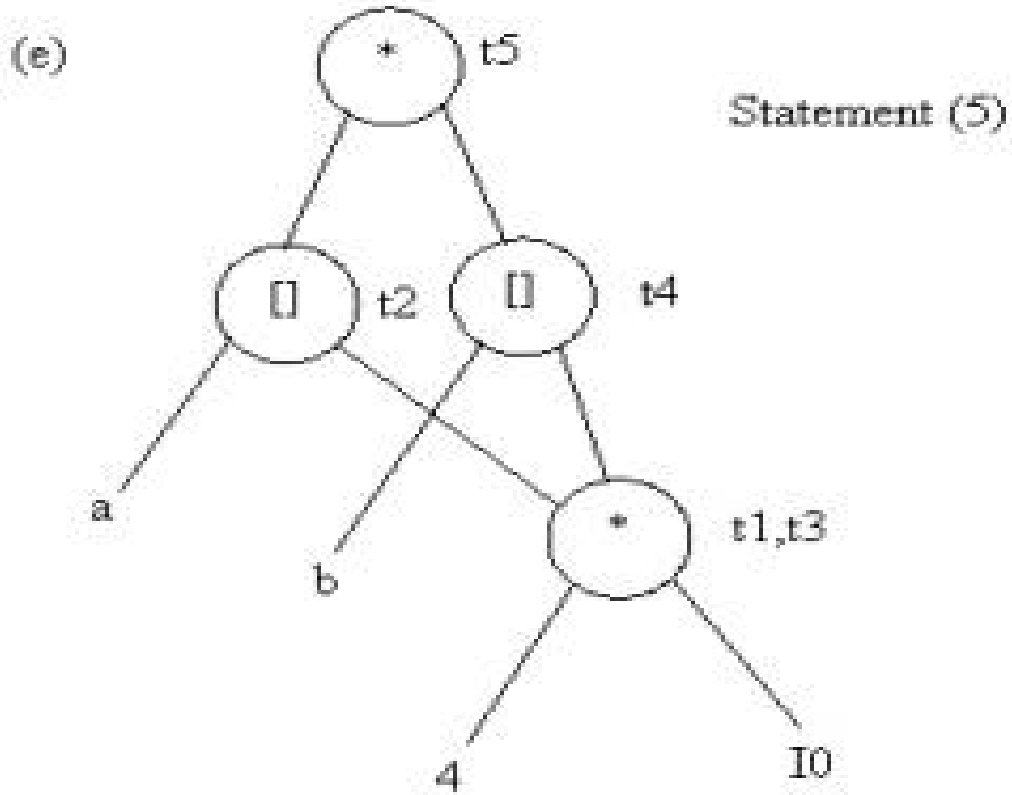
8. t7 := i+1

9. i := t7

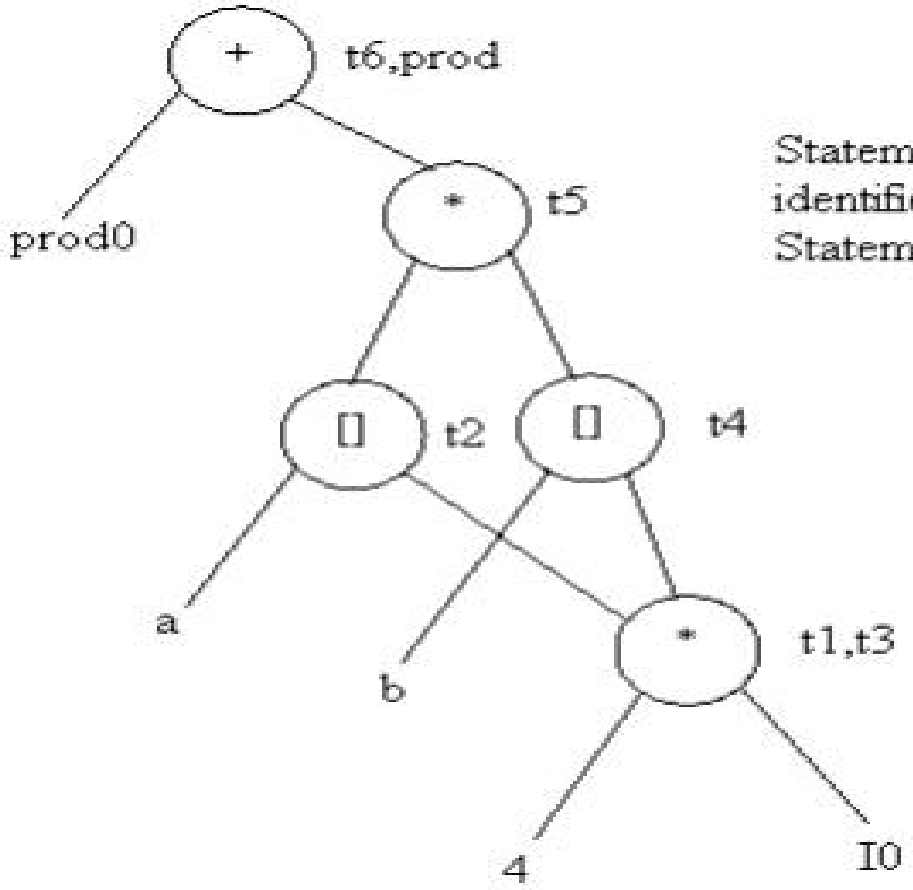
10. if i<=20 goto (1)



1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)

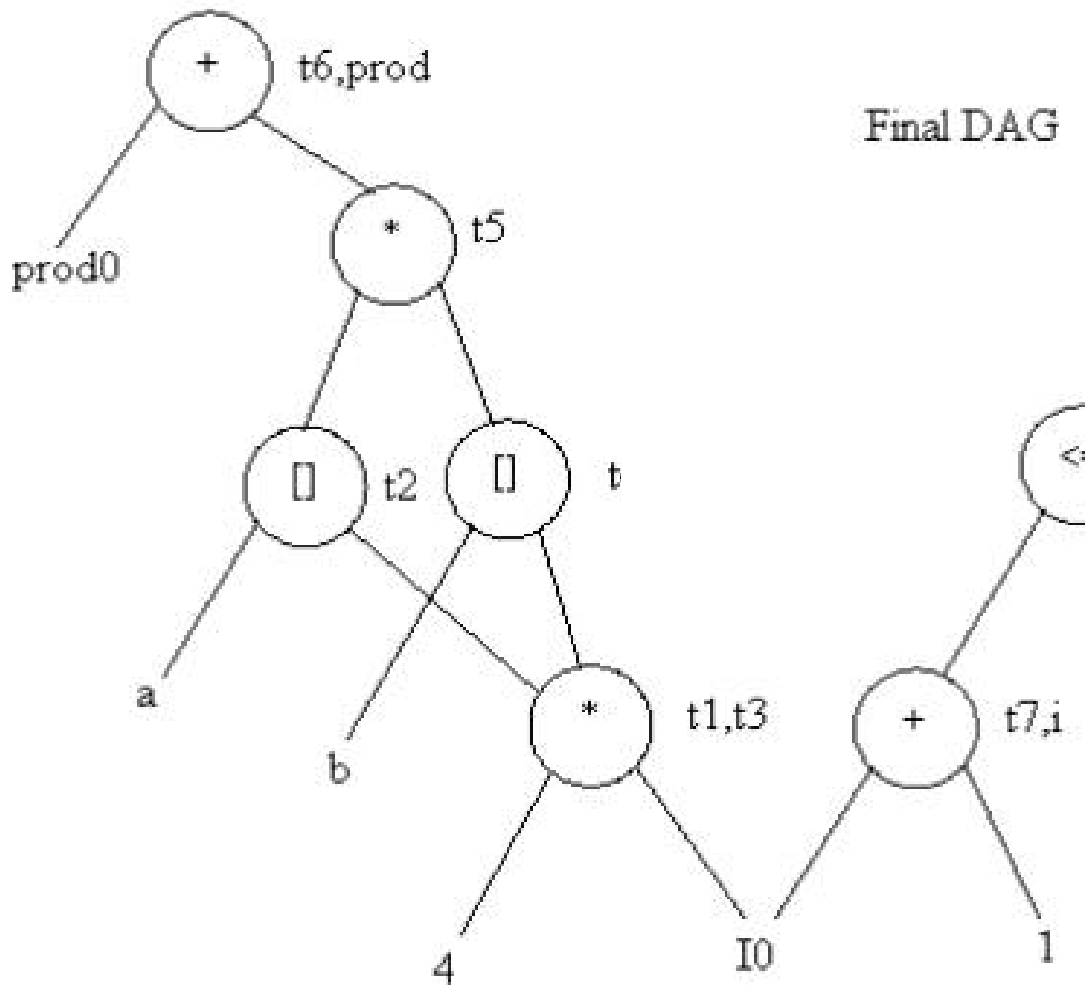


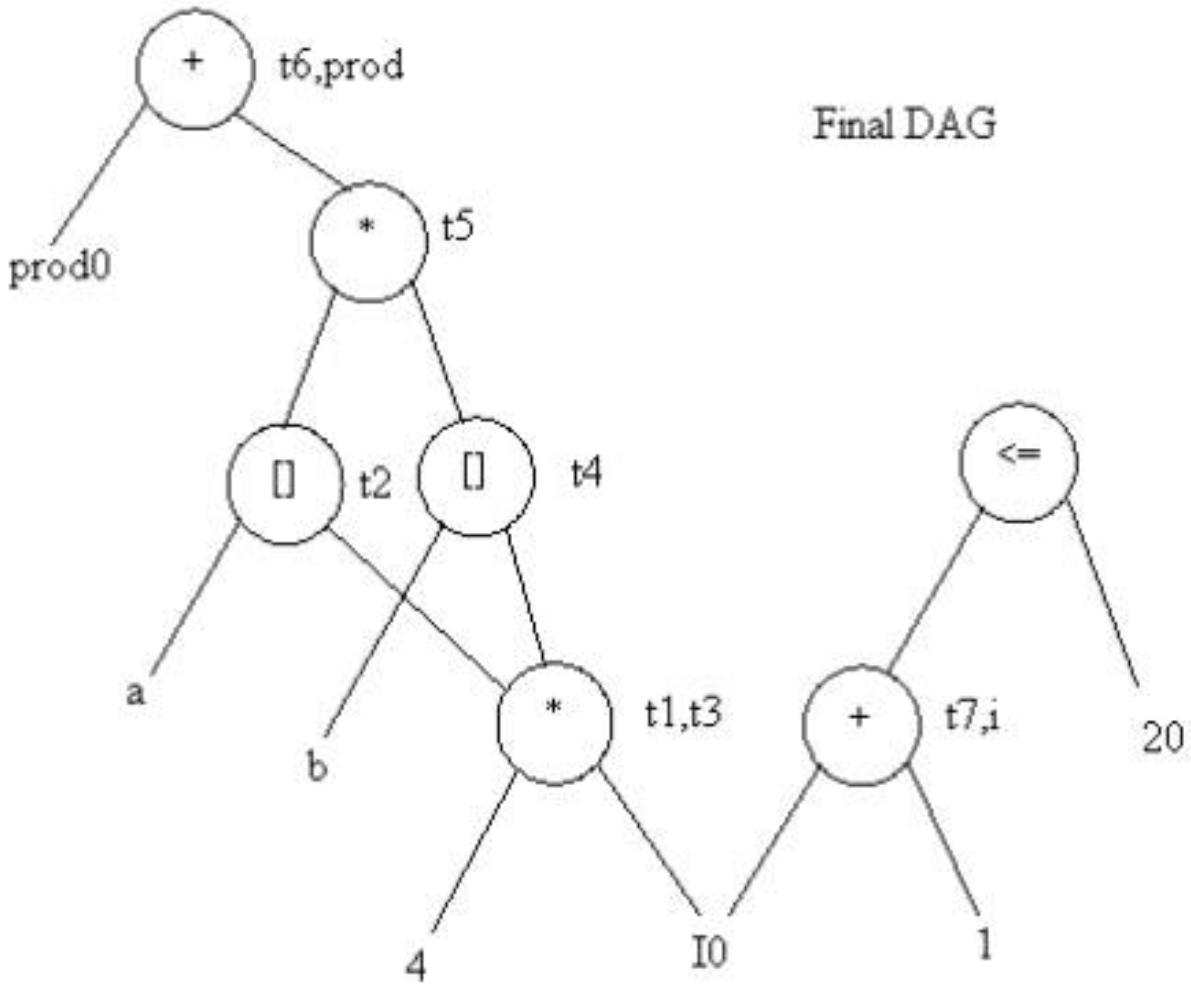
1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)



Statement (6), attach identifier prod for Statement (7)

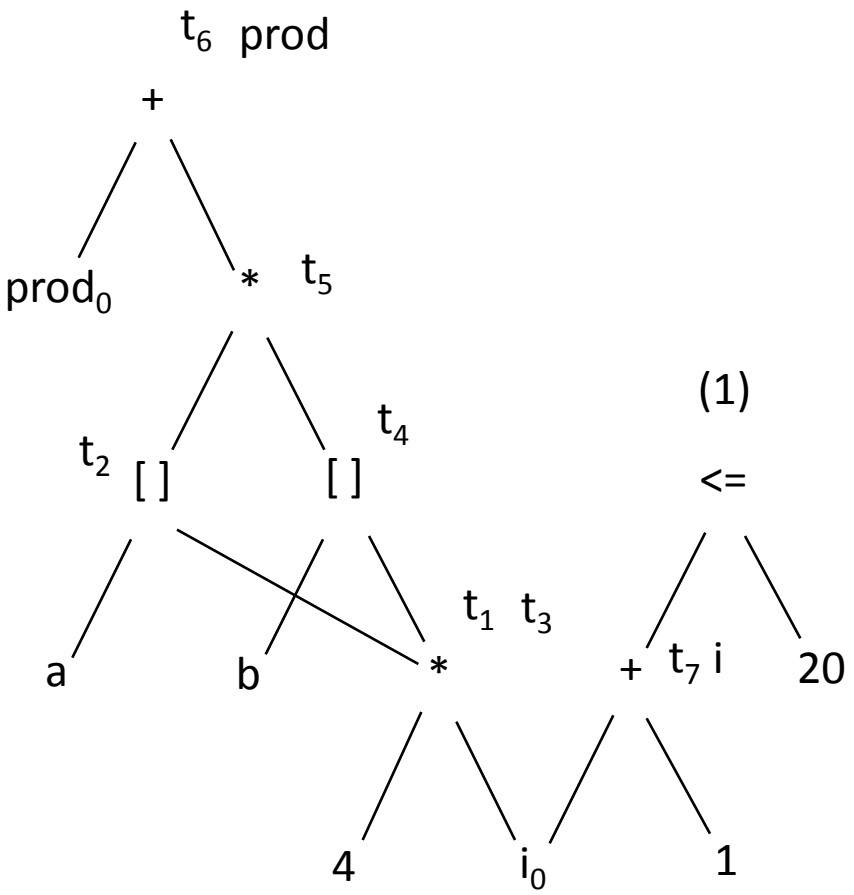
1. t1 := 4* i
2. t2 := a[t1]
3. t3 := 4* i
4. t4 := b[t3]
5. t5 := t2*t4
6. t6 := prod+t5
7. prod := t6
8. t7 := i+1
9. i := t7
10. if i<=20 goto (1)




Final DAG
+
t6,prod
prod0
*
t5
[]
t2
[]
t4
a
b
*
t1,t3
4
I0
<=
+
t7,i
20
1

DAG representation: example

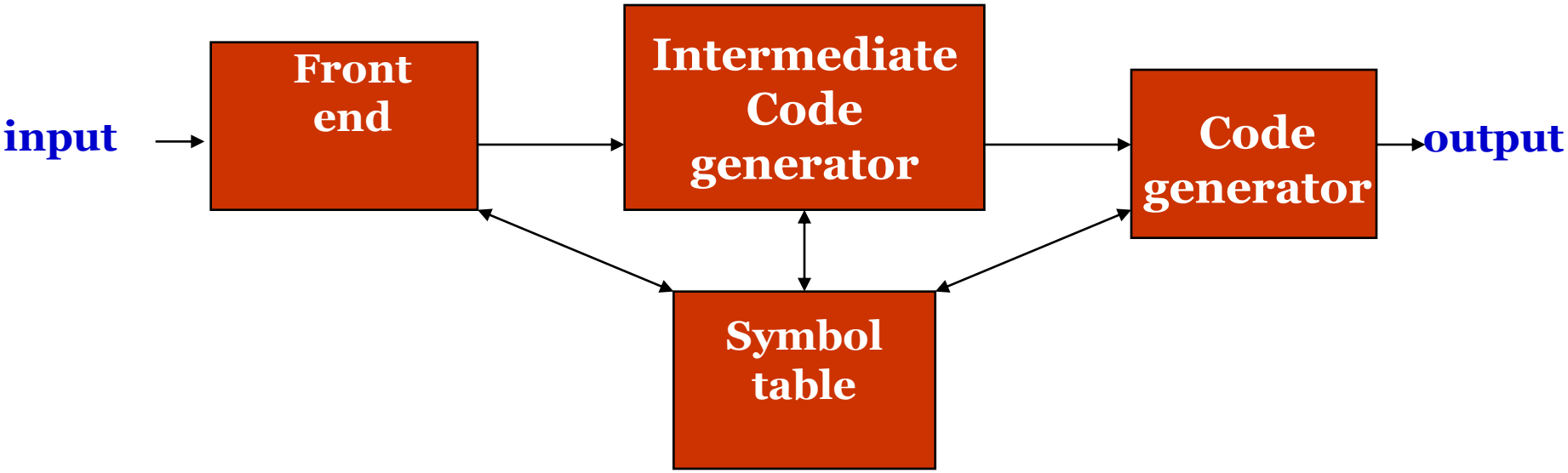
1. t_1 := 4 * i
2. t_2 := a[t_1]
3. t_3 := 4 * i
4. t_4 := b[t_3]
5. t_5 := t_2 * t_4
6. t_6 := prod + t_5
7. prod := t_6
8. t_7 := i + 1
9. i := t_7
10. if i <= 20 goto (1)



Target code Generation

D. VIJAYAKUMAR
Asst. Prof (Sr. Grade) / CSE
National Engineering College

Code generation and Instruction Selection



- output code must be correct
- output code must be of high quality
- code generator should run efficiently

Issues in the design of code generator

- **Input**: Intermediate representation with symbol table

  assume that input has been validated by the front end

- **target programs** :
  - absolute machine language
    fast for small programs
  - relocatable machine code
    requires linker and loader
  - assembly code
    requires assembler, linker, and loader

Instruction Selection

- Instruction selection
- uniformity
- completeness
- Instruction speed
- Register allocation
  Instructions with register operands are faster
  - store long life time and counters in registers
  - temporary locations
  - Even odd register pairs
- Evaluation order

Instruction Selection

- straight forward code if efficiency is not an issue

a=b+c	Mov b, R_0	
d=a+e	Add c, R_0	
	Mov R_0, a	
	Mov a, R_0	can be eliminated
	Add e, R_0	
	Mov R_0, d	
a=a+1	Mov a, R_0	Inc a
	Add #1, R_0	
	Mov R_0, a	

Target Machine

- Byte addressable with 4 bytes per word
- It has n registers R_0, R_1, ..., R_{n-l}
- Two address instructions of the form
  opcode source, destination
- Usual opcodes like move, add, sub etc.
- Addressing modes

MODE	FORM	ADDRESS
Absolute	M	M
register	R	R
index	c(R)	c+cont(R)
indirect register	*R	cont(R)
indirect index	*c(R)	cont(c+cont(R))
literal	#c	c

Code Generator

- consider each statement
- remember if operand is in a register
- **Register descriptor**
  - Keep track of what is currently in each register.
  - Initially all the registers are empty
- **Address descriptor**
  - Keep track of location where current value of the name can be found at runtime
  - The location might be a register, stack, memory address or a set of those

Code Generation Algorithm

for each X = Y op Z do

- invoke a function getreg to determine location L where X must be stored. Usually L is a register.

- Consult address descriptor of Y to determine Y'. Prefer a register for Y'. If value of Y not already in L generate

  Mov Y', L

- Generate op Z', L

  Again prefer a register for Z. Update address descriptor of X to indicate X is in L. If L is a register update its descriptor to indicate that it contains X and remove X from all other register descriptors.

- If current value of Y and/or Z have no next use and are dead on exit from block and are in registers, change register descriptor to indicate that they no longer contain Y and/or Z.

Function getreg

1. If Y is in register (that holds no other values) and Y is not live and has no next use after

   X = Y op Z

   then return register of Y for L.

2. Failing (1) return an empty register

3. Failing (2) if X has a next use in the block or op requires register then get a register R, store its content into M (by Mov R, M) and use it.

4. else select memory location X as L

Example

Stmt	code	reg desc	addr desc
t_1=a-b	mov a,R_0	R_0 contains t_1	t_1 in R_0
	sub b,R_0		
t_2=a-c	mov a,R_1	R_0 contains t_1	t_1 in R_0
	sub c,R_1	R_1 contains t_2	t_2 in R_1
t_3=t_1+t_2	add R_1,R_0	R_0 contains t_3	t_3 in R_0
		R_1 contains t_2	t_2 in R_1
d=t_3+t_2	add R_1,R_0	R_0 contains d	d in R_0
	mov R_0,d		d in R_0 and memory